Module 11 Lab - Hyperparameter Tuning & AutoML**Objective:** To learn how to optimize model performance by tuning **hyperparameters** and to get an introduction to the powerful concept of **Automated Machine Learning (AutoML).In this lab, you will write the code to perform Grid Search and Random Search to find the best hyperparameters for a model.**

Part 1: What are Hyperparameters?**Concept:** In machine learning, there are two types of parameters:1. **Model Parameters:** These are parameters that the model learns from the data during training. For example, the coefficients in a Linear Regression model.2. **Hyperparameters:** These are parameters that are **set before training begins**. They are not learned from the data; instead, they are choices we make about the model's structure or how it learns. * *Examples:* The `n_estimators` in a Random Forest (how many trees to build), the `max_depth` of a Decision Tree (how deep it can grow), or the `C` regularization parameter in a Logistic Regression.Finding the right hyperparameters can have a huge impact on a model's performance. **Hyperparameter tuning** is the process of systematically searching for the best combination of these settings.

Part 2: SetupWe will use the Iris dataset and a `RandomForestClassifier`, which has several important hyperparameters we can tune.

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load and prepare data
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,

# A baseline model with default hyperparameters
baseline_model = RandomForestClassifier(random_state=42)
baseline_model.fit(X_train, y_train)
y_pred_baseline = baseline_model.predict(X_test)
accuracy_baseline = accuracy_score(y_test, y_pred_baseline)
print(f"Accuracy of baseline Random Forest: {accuracy_baseline:.2%}")
```

```
Accuracy of baseline Random Forest: 100.00%
```

Part 3: Grid Search**Concept:** Grid Search is the most straightforward tuning method. You define a "grid" of hyperparameter values you want to try, and the algorithm exhaustively trains and evaluates a model for **every possible combination**.* **Pro:** It's guaranteed to find the best combination within the grid.* **Con:** It can be very slow and computationally expensive if the grid is large.

Task 1: Perform a Grid Search**Your Task:** Use `GridSearchCV` from `sklearn.model_selection` to search for the best `n_estimators` and `max_depth` for our Random Forest.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Define the grid of hyperparameters to search
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, None]
}

# 2. Create a GridSearchCV instance
grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5,
    n_jobs=-1,
    verbose=2
)

# 3. Fit the grid search to the data
grid_search.fit(X_train, y_train)

# 4. Print the best parameters and the best score
print(f"Best Parameters found by Grid Search: {grid_search.best_params_}")
print(f"Best cross-validated score: {grid_search.best_score_:.2%}")

# 5. (Optional) Evaluate the best model on the test set
best_grid_model = grid_search.best_estimator_
y_pred_grid = best_grid_model.predict(X_test)
accuracy_grid = accuracy_score(y_test, y_pred_grid)
print(f"Test accuracy of Grid Search best model: {accuracy_grid:.2%}")
```

```
Fitting 5 folds for each of 9 candidates, totalling 45 fits
Best Parameters found by Grid Search: {'max_depth': 5, 'n_estimators': 10
Best cross-validated score: 94.29%
Test accuracy of Grid Search best model: 100.00%
```

Part 4: Random Search**Concept:** Random Search is often more efficient than Grid Search. Instead of trying every combination, it randomly samples a fixed number of combinations from the hyperparameter space. * **Pro:** It's much faster and can explore a wider range of values.* **Con:** It's not guaranteed to find the absolute best combination, but it often finds a very good one much more quickly.

Task 2: Perform a Random Search**Your Task:** Use `RandomizedSearchCV` to perform a random search over a larger hyperparameter space.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

# 1. Define the distribution of hyperparameters to sample from
param_dist = {
    'n_estimators': randint(50, 300),
    'max_depth': [3, 5, 10, 15, 20, None],
    'min_samples_split': randint(2, 11),
    'min_samples_leaf': randint(1, 5)
}

# 2. Create a RandomizedSearchCV instance
random_search = RandomizedSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_distributions=param_dist,
    n_iter=20,          # number of random combinations to try
    cv=5,
    n_jobs=-1,
    verbose=2,
    random_state=42
)

# 3. Fit the random search to the data
random_search.fit(X_train, y_train)

# 4. Print the best parameters and the best score
print(f"Best Parameters found by Random Search: {random_search.best_para
print(f"Best cross-validated score: {random_search.best_score_:.2%}")

# 5. (Optional) Evaluate the best model on the test set
best_random_model = random_search.best_estimator_
y_pred_random = best_random_model.predict(X_test)
accuracy_random = accuracy_score(y_test, y_pred_random)
print(f"Test accuracy of Random Search best model: {accuracy_random:.2%}
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
Best Parameters found by Random Search: {'max_depth': 15, 'min_samples_le
Best cross-validated score: 94.29%
Test accuracy of Random Search best model: 100.00%
```

Part 5: Introduction to AutoML with AutoGluon**Concept:** AutoML takes hyperparameter tuning to the next level. It automates the entire ML workflow, including:* Data preprocessing* Feature engineering* Model selection (trying many different types of models)* Hyperparameter tuning* Ensemble creation**AutoGluon** is a popular and easy-to-use AutoML library. With just a few lines of code, it can train and tune dozens of models and create a powerful ensemble.**This part is fully coded.** Your task is to run it and see the power of AutoML. Note that it may take a few minutes to run.

```
# Install AutoGluon (run once; if Colab asks, restart the runtime afterw
!pip install autogluon

from autogluon.tabular import TabularPredictor

# AutoGluon requires the data in a single DataFrame with the target colu
train_data_ag = pd.DataFrame(X_train, columns=iris.feature_names)
train_data_ag['species'] = y_train

# Create a test DataFrame as well
test_data_ag = pd.DataFrame(X_test, columns=iris.feature_names)
test_data_ag['species'] = y_test

# Create and train the TabularPredictor (runs for up to 60 seconds)
predictor = TabularPredictor(label='species', eval_metric='accuracy').fi
    train_data=train_data_ag,
    time_limit=60
)

# Evaluate the predictor on the test data
leaderboard = predictor.leaderboard(test_data_ag)
print(leaderboard)
```

```
        Fitting with cpus=1, gpus=0, mem=0.3/10.5 GB
        0.9524   = Validation score   (accuracy)
        4.87s    = Training   runtime
        0.0s     = Validation runtime
Fitting model: LightGBMLarge ... Training model for up to 45.50s of the 4
        Fitting with cpus=1, gpus=0, mem=0.2/10.5 GB
        0.8571   = Validation score   (accuracy)
        0.52s    = Training   runtime
```

```
        0.0s     = Validation runtime
Fitting model: WeightedEnsemble_L2 ... Training model for up to 59.91s of
        Fitting 1 model on all data | Fitting with cpus=2, gpus=0, mem=0.
        Ensemble Weights: {'LightGBMXT': 1.0}
        0.9524   = Validation score   (accuracy)
        0.08s    = Training   runtime
        0.0s     = Validation runtime
AutoGluon training complete, total runtime = 16.34s ... Best model: Weigh
TabularPredictor saved. To load, use: predictor = TabularPredictor.load("
                  model  score_test  score_val eval_metric  pred_time_tes
0               XGBoost    1.000000   0.952381    accuracy        0.01720
1      RandomForestGini    1.000000   0.857143    accuracy        0.06577
2        ExtraTreesGini    1.000000   0.904762    accuracy        0.06883
3      RandomForestEntr    1.000000   0.857143    accuracy        0.07106
4        ExtraTreesEntr    1.000000   0.904762    accuracy        0.08316
5         LightGBMLarge    0.977778   0.857143    accuracy        0.00190
6               LightGBM    0.955556   0.857143    accuracy        0.00176
7        NeuralNetTorch    0.955556   0.952381    accuracy        0.00723
8            LightGBMXT    0.933333   0.952381    accuracy        0.00244
9              CatBoost    0.933333   0.952381    accuracy        0.00363
10  WeightedEnsemble_L2    0.933333   0.952381    accuracy        0.00426
11      NeuralNetFastAI    0.866667   0.904762    accuracy        0.02307

    pred_time_val  fit_time  pred_time_test_marginal  pred_time_val_margi
0        0.002625  0.144437                 0.017202                0.002
1        0.054963  0.570034                 0.065778                0.054
2        0.065752  0.534241                 0.068830                0.065
3        0.065965  0.545776                 0.071066                0.065
4        0.055476  0.555206                 0.083166                0.055
5        0.001586  0.519379                 0.001908                0.001
6        0.001501  0.300268                 0.001760                0.001
7        0.004491  4.866766                 0.007234                0.004
8        0.001588  3.553415                 0.002445                0.001
9        0.000616  1.039949                 0.003632                0.000
10       0.002672  3.638327                 0.001817                0.001
11       0.009005  1.900613                 0.023078                0.009

    fit_time_marginal  stack_level  can_infer  fit_order
0            0.144437            1       True          9
1            0.570034            1       True          4
2            0.534241            1       True          7
3            0.545776            1       True          5
4            0.555206            1       True          8
5            0.519379            1       True         11
6            0.300268            1       True          3
7            4.866766            1       True         10
8            3.553415            1       True          2
9            1.039949            1       True          6
10           0.084913            2       True         12
11           1.900613            1       True          1
```

📝 Knowledge Check**Instructions:** Answer the following

questions in this markdown cell.1. **What is the main difference between a model parameter and a hyperparameter?**2. *_When would you choose to use Grid Search over Random Search, and vice-versa?*_3. ***Looking at the AutoGluon leaderboard, which model performed the best? What does AutoML do that makes it so powerful compared to manual tuning?***_[ENTER YOUR ANSWERS HERE]_*

1. What is the main difference between a model parameter and a hyperparameter?

Model parameters are learnt from the data during training, such as the coefficients in a Linear Regression model. Hyperparameters are set before training begins and are not learned from the data. They are choices about the model's structure or how it learns, such as `n_estimators` and `max_depth` in a Random Forest.

2. When would you choose to use Grid Search over Random Search, and vice-versa?

Use Grid Search when the hyperparameter space is small and one wants to test every combination, since it is guaranteed to find the best combination within the grid. Use Random Search when the space is large or has many hyperparameters, because it samples a fixed number of combinations, runs much faster, and can explore a wider range of values. There is no guarantee to find the absolute best combination, but it often finds a good one much more quickly.

3. Looking at the AutoGluon leaderboard, which model performed the best? What does AutoML do that makes it so powerful compared to manual tuning?

On the test set, the best performing model on the AutoGluon leaderboard was XGBoost with a test accuracy of 100% and a validation accuracy of 95.24%. It tied with four other models RandomForestGini, ExtraTreesGini, RandomForestEntr, and ExtraTreesEntr, all of which reached 100% test accuracy. I am a bit skeptical about the 100% accuracy. Because the dataset is small and its classes are well separated, perhaps it is common for models to reach 100% test accuracy, and so this leaderboard cannot clearly separate them. AutoGluon's own log named WeightedEnsemble_L2 as the best model because it selects based on validation score, but that ensemble scored 93.33% on the test set.

Since the Iris test set is only 45 samples, these differences amount to just a few predictions. I believe providing another dataset could help ameliorate my skepticism.

AutoML is powerful compared to manual tuning because it automates the entire ML workflow. In this lab, manual tuning only searched one model type, a Random Forest. Grid Search tested 9 combinations (45 fits) and Random Search tested 20 combinations (100 fits). Both ended up at the same 94.29% cross-validated score with 100% test accuracy. Within ~16 seconds of training, AutoGluon trained 11 different models, including XGBoost, LightGBM, CatBoost, Random Forests, Extra Trees, and neural networks. Then, it combined them into a weighted ensemble. It handles preprocessing, model selection, hyperparameter tuning, and ensembling with a few lines of code, whereas manual tuning means picking one model type and testing it at a time.